

Linux Application Development - Day 5

Threading, Networking & Sockets

Instructor: Linux Systems Expert

Duration: Full Day

Focus: Concurrent Programming & Network Communication

Welcome to Day 5!

Days 1-4 Recap

- **Day 1:** Build tools, GCC, libraries, Make, Git, IDEs
- **Day 2:** Debugging (GDB, Valgrind), system calls, memory management
- **Day 3:** File systems, VFS, file I/O, mmap, locking, epoll
- **Day 4:** Processes, fork/exec, IPC mechanisms, signals, daemons

Today: Threading and network programming - the final frontier!

Today's Agenda

Morning Session (9:00 AM - 12:00 PM)

1. POSIX Threads (pthreads) - creation and lifecycle
2. Thread synchronization - mutexes, condition variables
3. Thread safety and race conditions
4. Deadlock prevention strategies

Afternoon Session (1:00 PM - 5:00 PM)

5. Socket programming - TCP and UDP
6. Client-server architecture
7. I/O multiplexing (select/poll/epoll for sockets)
8. Concurrent server patterns

Section 1: POSIX Threads

Multi-threading on Linux

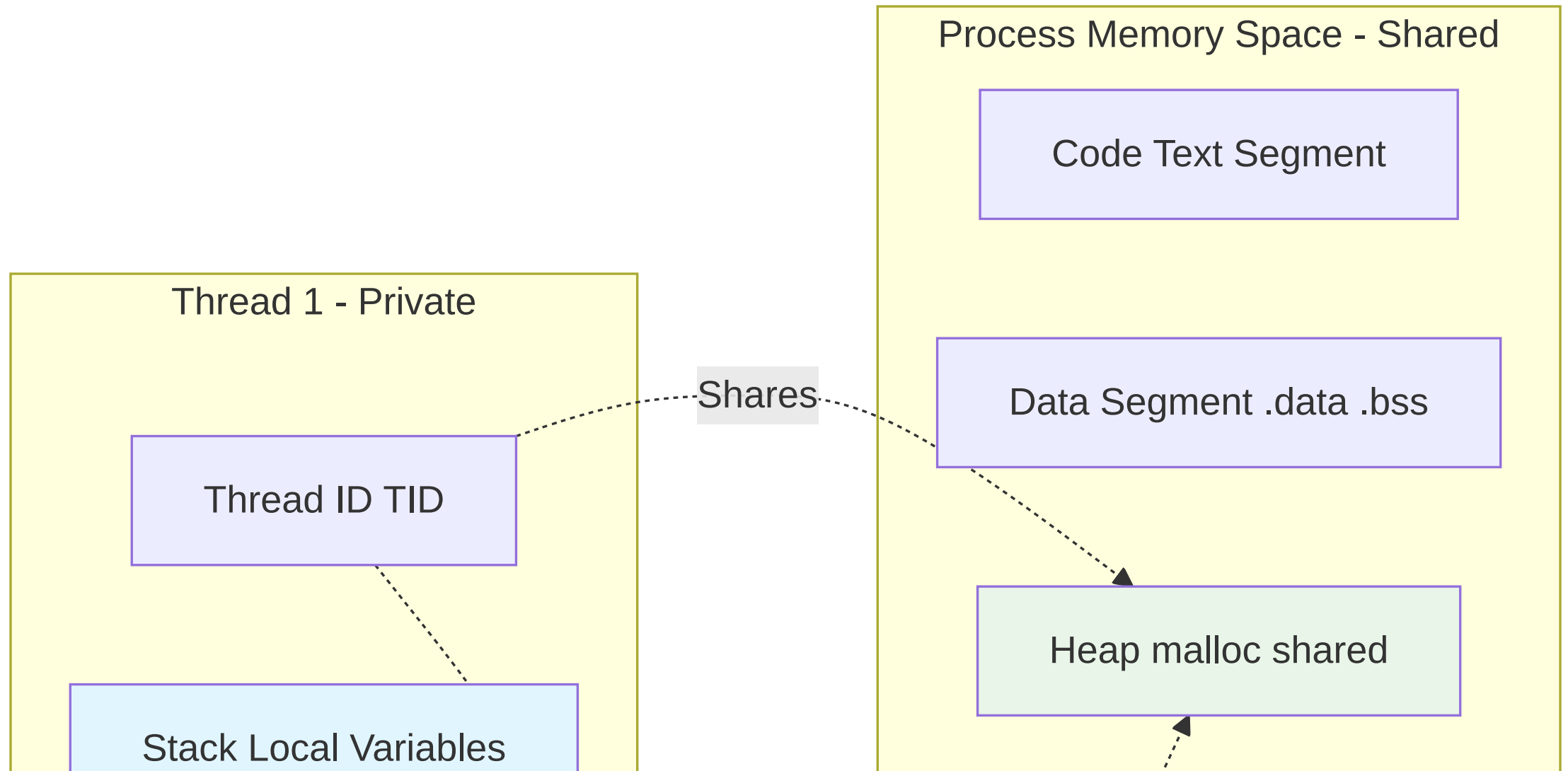
NPTL: Native POSIX Thread Library (since Linux 2.6)

- 1:1 threading model (one kernel thread per user thread)
- Efficient, scalable to 10,000+ threads
- Part of glibc

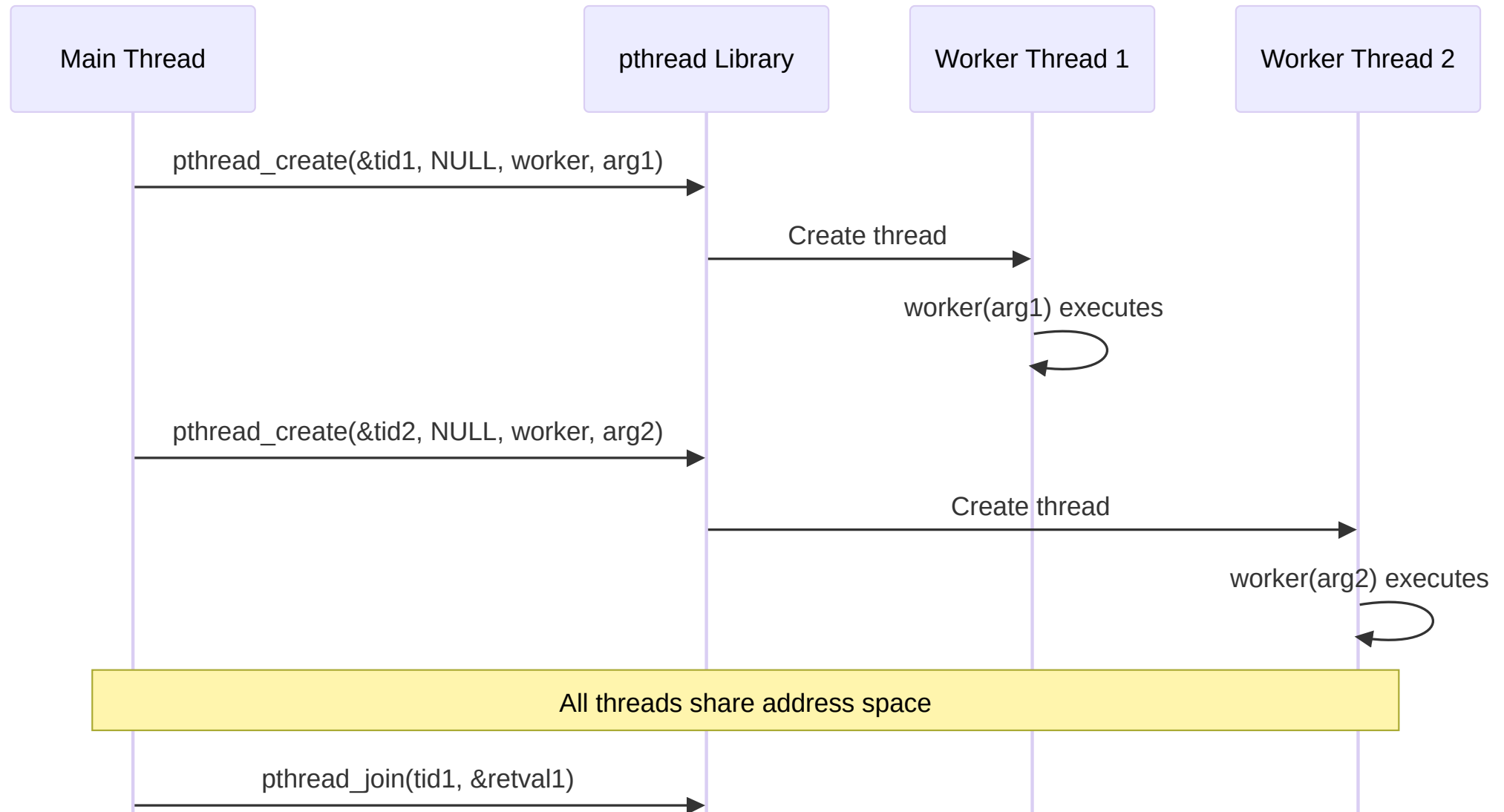
Why Use Threads?

- Shared memory between threads (no IPC needed)
- Faster creation than fork (~10 μ s vs ~1 ms)
- Lighter context switches
- Natural for I/O-bound and GUI applications

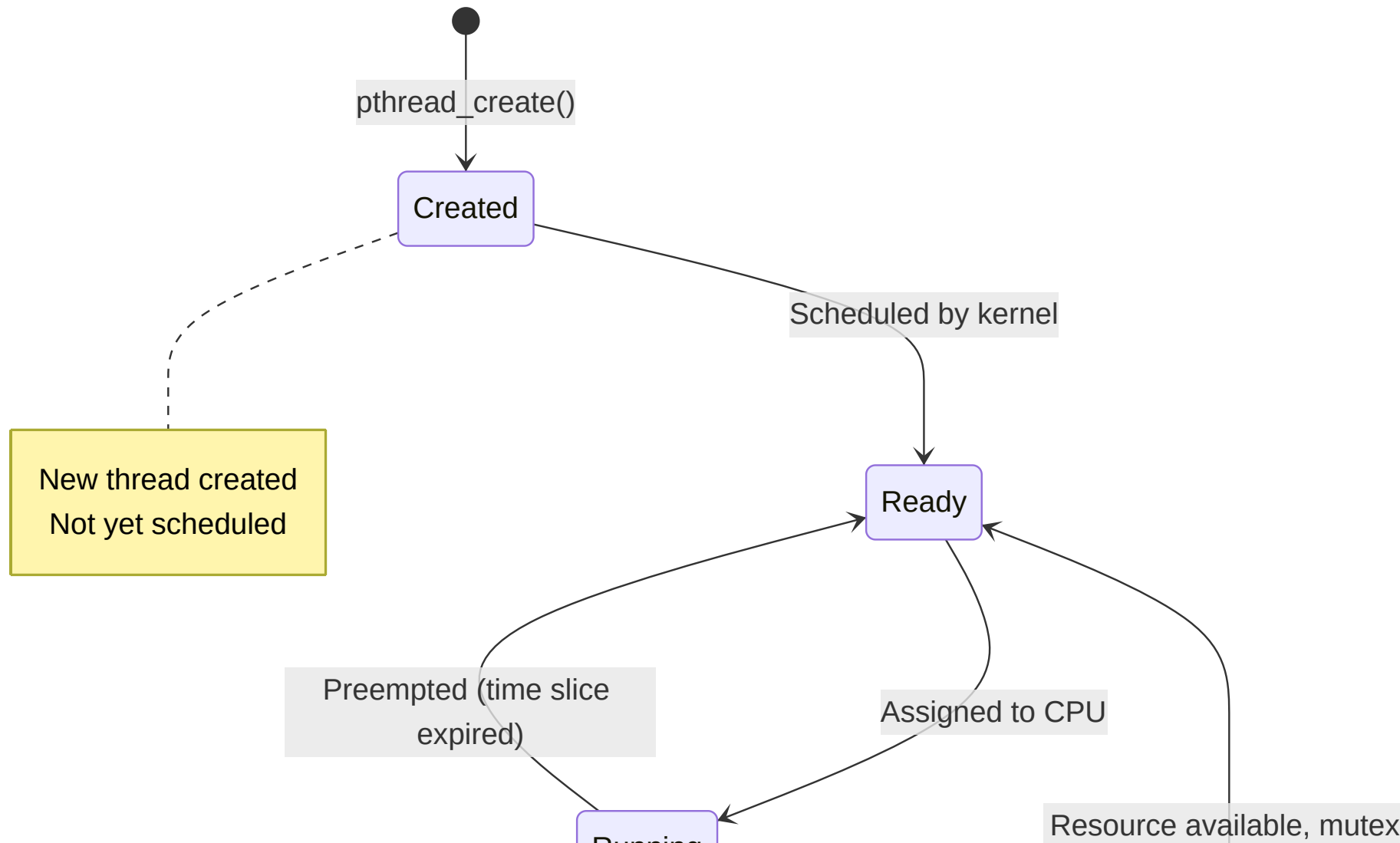
Thread Memory Layout



Thread Lifecycle



Thread States



Creating Threads

```
#include <pthread.h>
#include <stdio.h>

void *worker_func(void *arg) {
    int id = *(int *)arg;
    printf("Thread %d running\n", id);
    return NULL;
}

int main() {
    pthread_t tid1, tid2;
    int id1 = 1, id2 = 2;

    // Create threads
    pthread_create(&tid1, NULL, worker_func, &id1);
    pthread_create(&tid2, NULL, worker_func, &id2);

    // Wait for completion
    pthread_join(tid1, NULL);
    pthread_join(tid2, NULL);

    return 0;
}
```


pthread_create() Parameters

```
int pthread_create(pthread_t *thread,  
                  const pthread_attr_t *attr,  
                  void *(*start_routine)(void *),  
                  void *arg);
```

thread: Pointer to pthread_t to store thread ID

attr: Thread attributes (NULL for defaults)

start_routine: Function pointer for thread to execute

arg: Argument passed to start_routine

Returns: 0 on success, error number on failure

Thread Attributes

```
pthread_attr_t attr;  
pthread_attr_init(&attr);  
  
// Set stack size to 2 MB  
pthread_attr_setstacksize(&attr, 2 * 1024 * 1024);  
  
// Set as detached thread  
pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_DETACHED);  
  
pthread_create(&tid, &attr, worker, NULL);  
pthread_attr_destroy(&attr);
```

Detached threads: Cannot be joined, resources freed immediately on exit

Joinable threads: Must call `pthread_join()` to avoid memory leaks

Detaching Threads

```
void *worker(void *arg) {  
    // Do work  
    return NULL;  
}  
  
int main() {  
    pthread_t tid;  
    pthread_create(&tid, NULL, worker, NULL);  
  
    // Option 1: Detach from main thread  
    pthread_detach(tid);  
  
    // Option 2: Self-detach in worker  
    // Inside worker: pthread_detach(pthread_self());  
  
    // Main can exit without join  
    sleep(1); // Give worker time to finish  
    return 0;  
}
```

Passing Arguments to Threads

```
// WRONG! Local variable address passed
for (int i = 0; i < 5; i++) {
    pthread_create(&tids[i], NULL, worker, &i); // RACE!
}
```

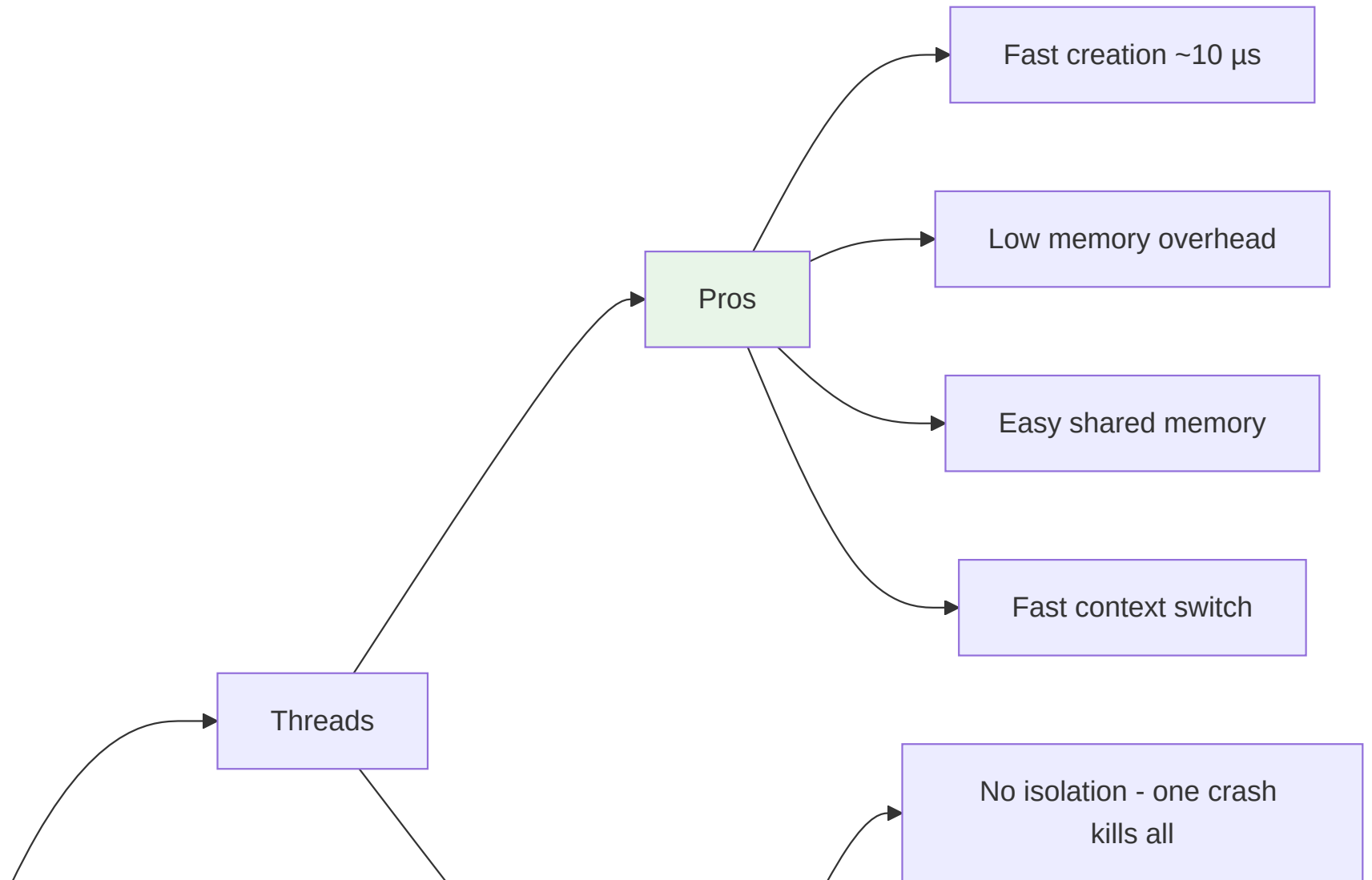
```
// RIGHT! Each thread gets unique storage
int ids[5];
for (int i = 0; i < 5; i++) {
    ids[i] = i;
    pthread_create(&tids[i], NULL, worker, &ids[i]);
}
```

```
// Or use dynamic allocation
for (int i = 0; i < 5; i++) {
    int *id = malloc(sizeof(int));
    *id = i;
    pthread_create(&tids[i], NULL, worker, id);
    // Must free in worker function
}
```

Getting Return Values

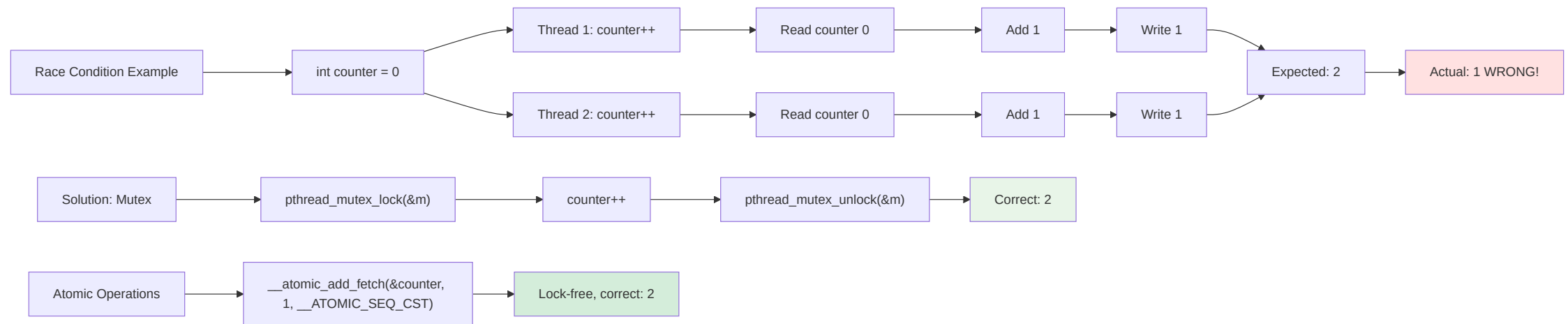
```
void *worker(void *arg) {  
    int *result = malloc(sizeof(int));  
    *result = 42;  
    return result; // Or: pthread_exit(result);  
}  
  
int main() {  
    pthread_t tid;  
    void *ret_val;  
  
    pthread_create(&tid, NULL, worker, NULL);  
    pthread_join(tid, &ret_val);  
  
    printf("Worker returned: %d\n", *(int *)ret_val);  
    free(ret_val);  
  
    return 0;  
}
```

Threads vs Processes



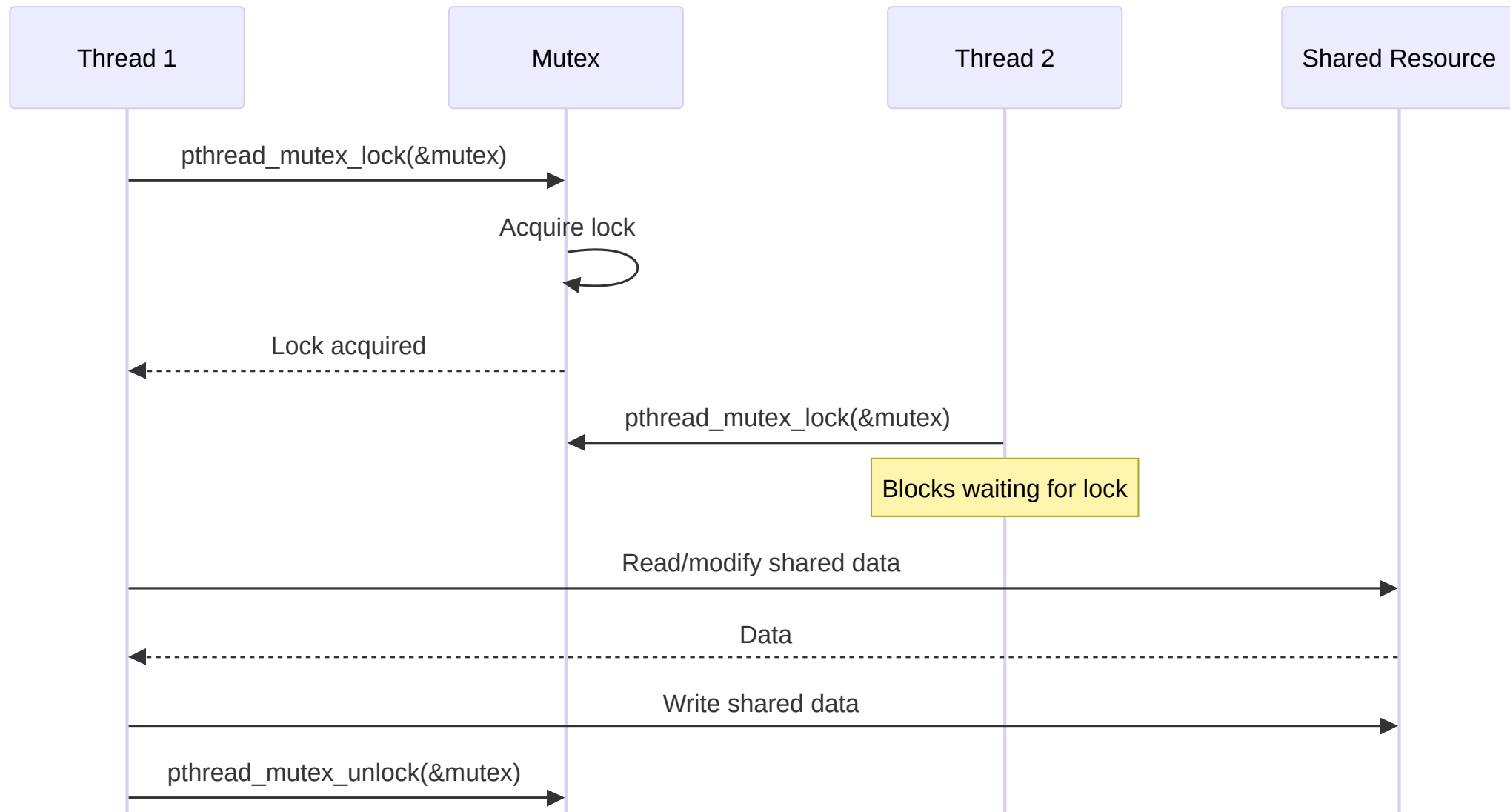
Section 2: Thread Synchronization

The Problem: Race Conditions



Without synchronization, concurrent access to shared data causes **data races**.

Mutexes (Mutual Exclusion)



Using Mutexes

```
#include <pthread.h>

pthread_mutex_t counter_mutex = PTHREAD_MUTEX_INITIALIZER;
int counter = 0;

void *increment(void *arg) {
    for (int i = 0; i < 100000; i++) {
        pthread_mutex_lock(&counter_mutex);
        counter++; // Critical section
        pthread_mutex_unlock(&counter_mutex);
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Counter: %d\n", counter); // Always 200000
    pthread_mutex_destroy(&counter_mutex);
    return 0;
}
```

Dynamic Mutex Initialization

```
pthread_mutex_t mutex;  
  
// Initialize with default attributes  
pthread_mutex_init(&mutex, NULL);  
  
// Use the mutex  
pthread_mutex_lock(&mutex);  
// ... critical section ...  
pthread_mutex_unlock(&mutex);  
  
// Clean up  
pthread_mutex_destroy(&mutex);
```

Static initialization: PTHREAD_MUTEX_INITIALIZER

Dynamic initialization: pthread_mutex_init() for heap-allocated mutexes

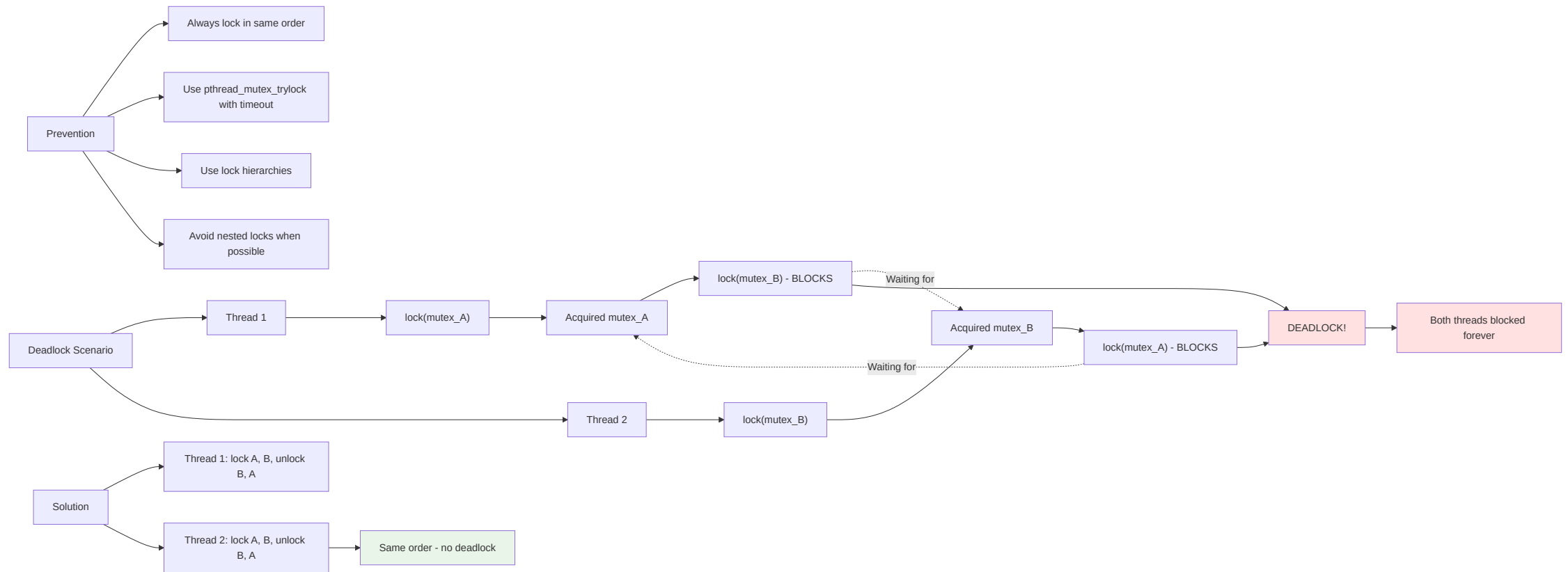
Mutex Attributes

```
pthread_mutexattr_t attr;  
pthread_mutex_t mutex;  
  
pthread_mutexattr_init(&attr);  
  
// Recursive mutex (same thread can lock multiple times)  
pthread_mutexattr_settype(&attr, PTHREAD_MUTEX_RECURSIVE);  
  
// Error-checking mutex (detects deadlocks)  
pthread_mutexattr_settype(&attr, PTHREAD_MUTEX_ERRORCHECK);  
  
pthread_mutex_init(&mutex, &attr);  
pthread_mutexattr_destroy(&attr);
```

RECURSIVE: Allows same thread to lock multiple times (must unlock same number)

ERRORCHECK: Returns EDEADLK if thread tries to relock

Deadlocks



Deadlock: Two threads waiting for each other's locks forever.

Deadlock Prevention

1. Always lock in the same order

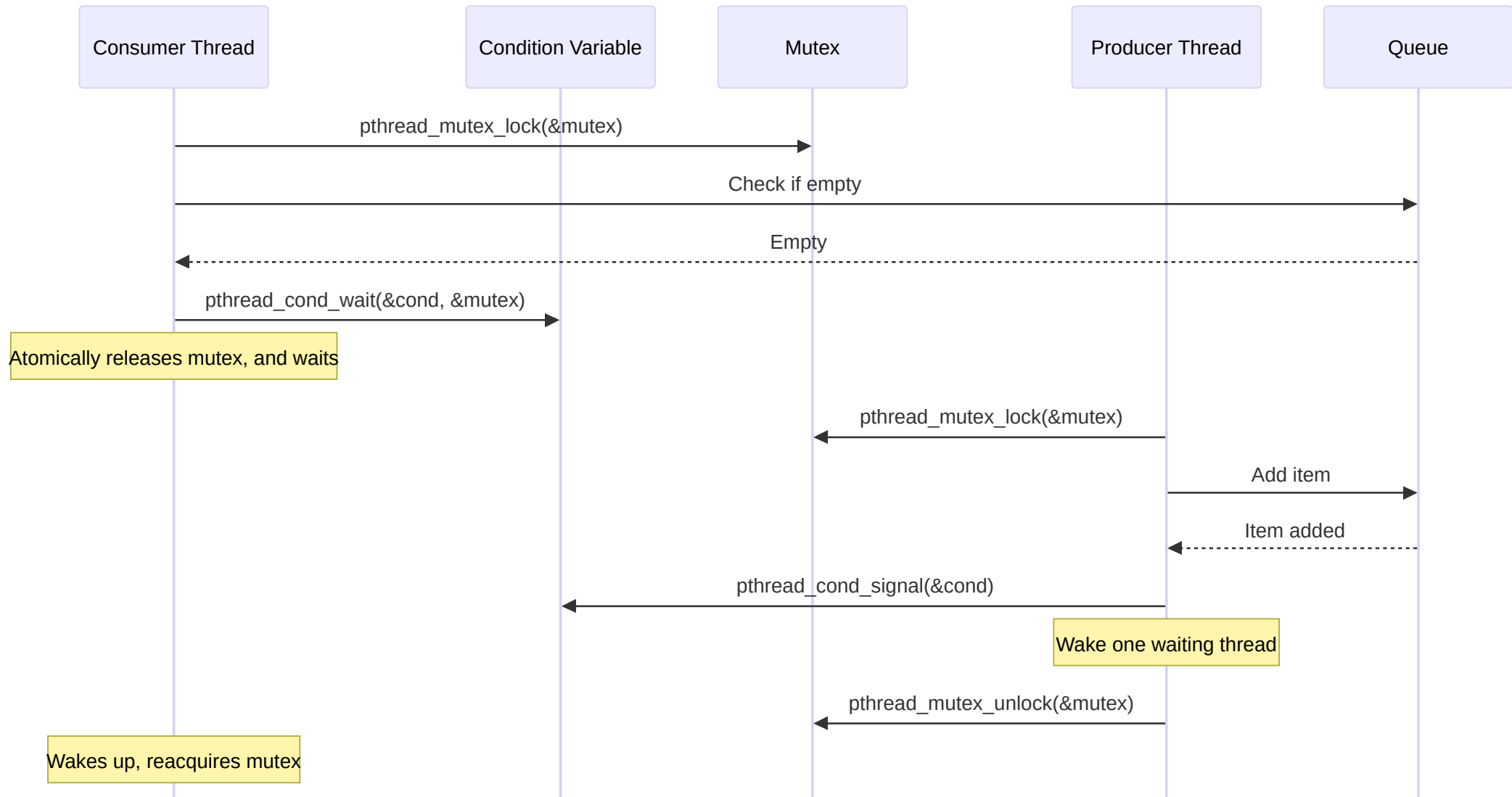
```
// Both threads lock A then B
pthread_mutex_lock(&mutex_A);
pthread_mutex_lock(&mutex_B);
// ... critical section ...
pthread_mutex_unlock(&mutex_B);
pthread_mutex_unlock(&mutex_A);
```

2. Use pthread_mutex_trylock() with timeout

```
while (pthread_mutex_trylock(&mutex_B) != 0) {
    pthread_mutex_unlock(&mutex_A);
    usleep(100); // Back off
    pthread_mutex_lock(&mutex_A);
}
```

3. Use lock hierarchies (assign numbers, always lock lower first)

Condition Variables



Producer-Consumer with Condition Variable

```
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
int queue[100];
int count = 0;

void *consumer(void *arg) {
    while (1) {
        pthread_mutex_lock(&mutex);

        // Wait while queue empty
        while (count == 0) {
            pthread_cond_wait(&cond, &mutex);
            // Atomically releases mutex and waits
            // On wakeup, reacquires mutex
        }

        // Consume item
        int item = queue[--count];
        pthread_mutex_unlock(&mutex);

        process(item);
    }
}
```

Producer Side

```
void *producer(void *arg) {  
    while (1) {  
        int item = produce_item();  
  
        pthread_mutex_lock(&mutex);  
  
        // Wait if queue full  
        while (count == 100) {  
            pthread_cond_wait(&cond, &mutex);  
        }  
  
        // Add item  
        queue[count++] = item;  
  
        // Signal one waiting consumer  
        pthread_cond_signal(&cond);  
  
        pthread_mutex_unlock(&mutex);  
    }  
}
```

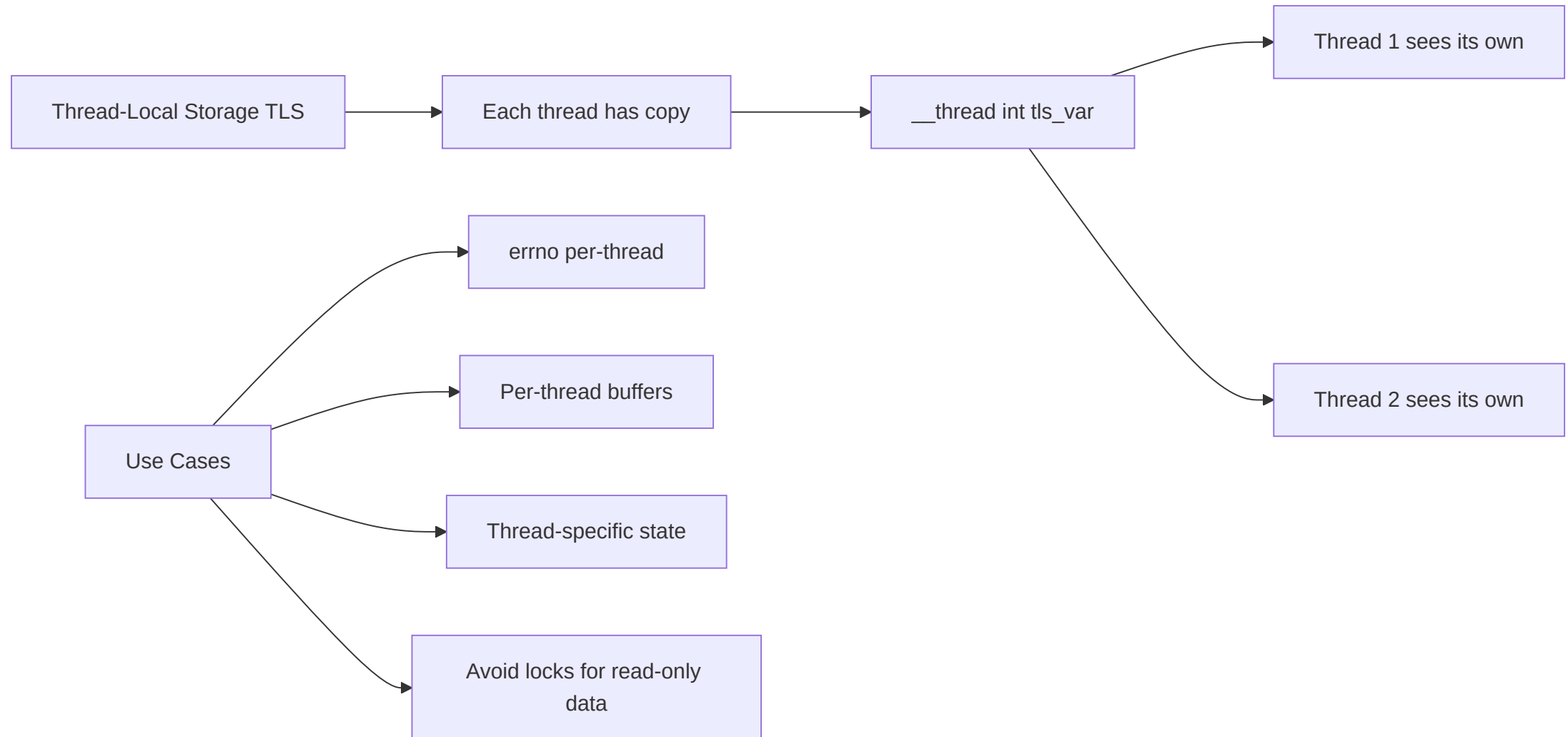

Spurious Wakeups

```
// WRONG! Check condition with if
if (count == 0) {
    pthread_cond_wait(&cond, &mutex);
}

// RIGHT! Check condition with while
while (count == 0) {
    pthread_cond_wait(&cond, &mutex);
}
```

Spurious wakeup: Thread may wake even without signal
Always use while loop to recheck condition after waking.

Thread-Local Storage (TLS)



Using TLS

```
// Static TLS with __thread keyword
__thread int tls_counter = 0;

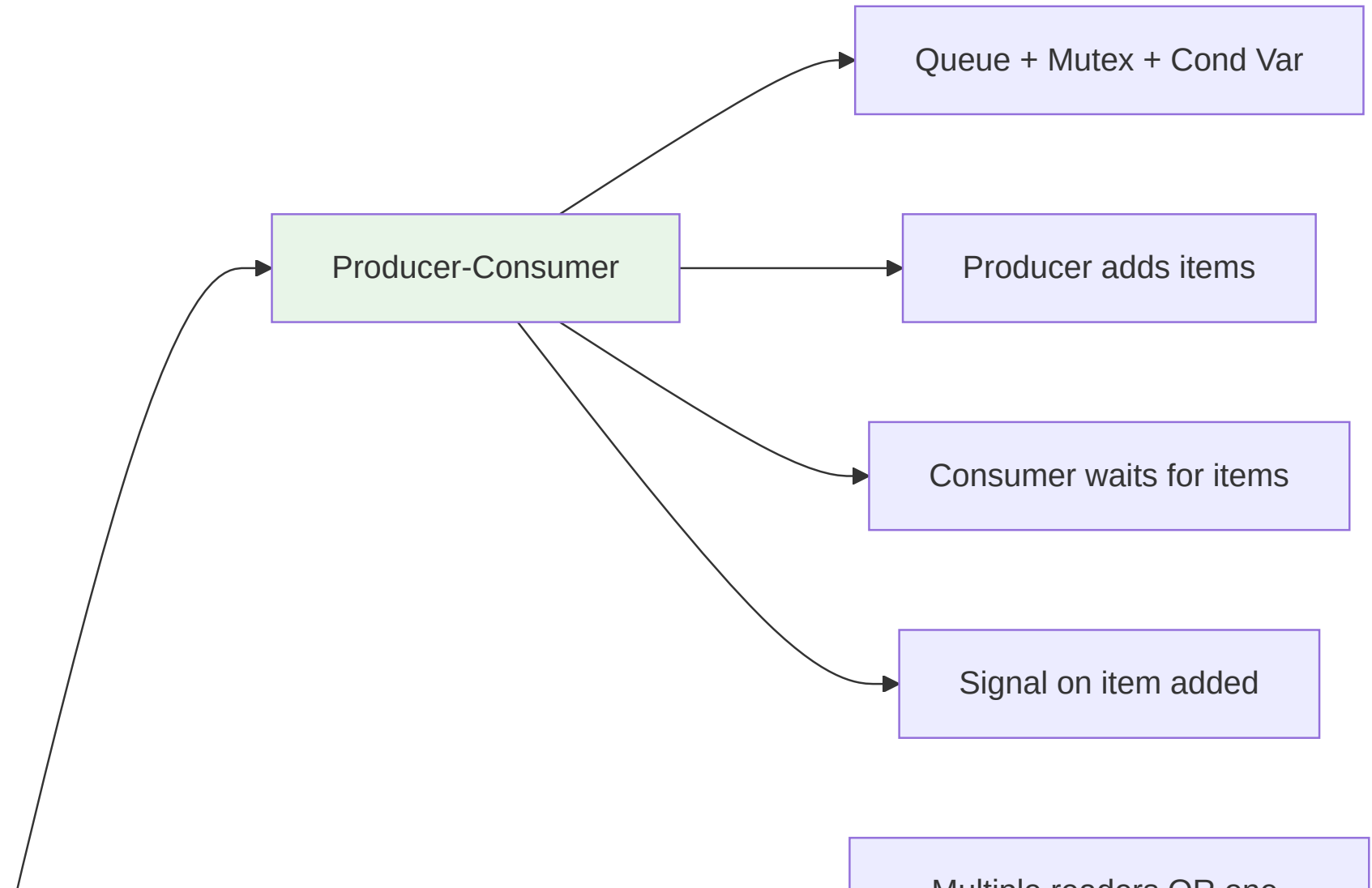
void *worker(void *arg) {
    for (int i = 0; i < 5; i++) {
        tls_counter++; // Each thread has own copy
    }
    printf("Thread %ld: %d\n", pthread_self(), tls_counter);
    return NULL;
}

// Dynamic TLS with pthread_key_t
pthread_key_t key;

void thread_data_destructor(void *data) {
    free(data);
}

pthread_key_create(&key, thread_data_destructor);
pthread_setspecific(key, malloc(100));
void *data = pthread_getspecific(key);
```

Thread Synchronization Patterns



Reader-Writer Locks

```
pthread_rwlock_t rwlock = PTHREAD_RWLOCK_INITIALIZER;
int shared_data = 0;

void *reader(void *arg) {
    pthread_rwlock_rdlock(&rwlock); // Shared read access
    int value = shared_data;
    pthread_rwlock_unlock(&rwlock);
    return NULL;
}

void *writer(void *arg) {
    pthread_rwlock_wrlock(&rwlock); // Exclusive write access
    shared_data++;
    pthread_rwlock_unlock(&rwlock);
    return NULL;
}
```

Multiple readers can hold lock simultaneously.

Atomic Operations

```
#include <stdatomic.h>

atomic_int counter = 0;

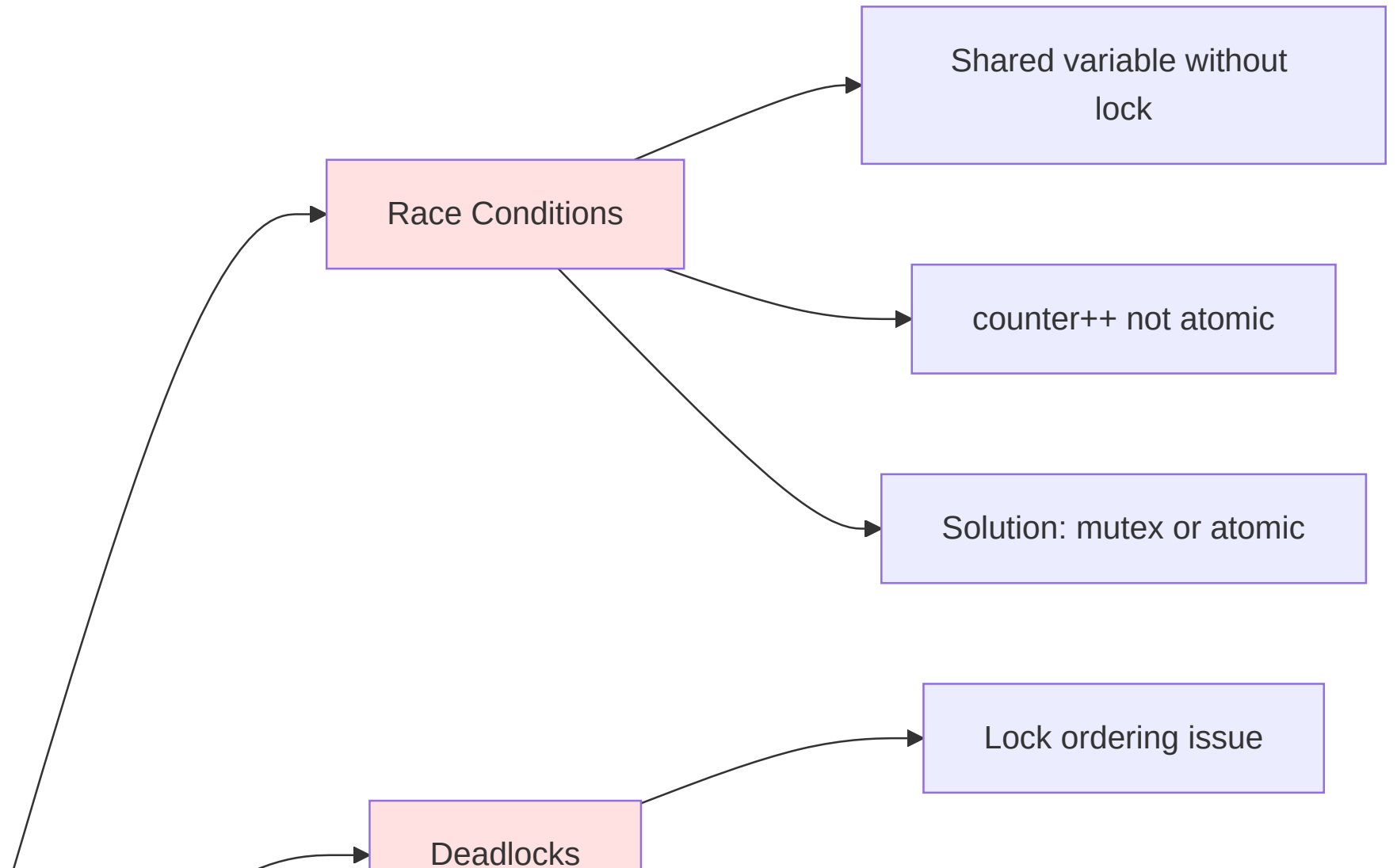
void *worker(void *arg) {
    for (int i = 0; i < 100000; i++) {
        // Lock-free atomic increment
        atomic_fetch_add(&counter, 1);
        // Or: __atomic_add_fetch(&counter, 1, __ATOMIC_SEQ_CST);
    }
    return NULL;
}
```

Atomic operations: Lock-free, faster for simple counters

GCC built-ins: `__atomic_*` and `__sync_*` functions

C11 stdatomic.h: More portable

Common Threading Pitfalls



Thread-Safe vs Non-Thread-Safe Functions

Non-thread-safe (use `_r` variants):

```
// NOT thread-safe
char *token = strtok(str, " ");
struct tm *tm = localtime(&time);
char *str = asctime(tm);

// Thread-safe alternatives
char *token = strtok_r(str, " ", &saveptr);
struct tm tm;
localtime_r(&time, &tm);
char buf[26];
asctime_r(&tm, buf);
```

Check man page for function thread-safety: "MT-Safe" or "MT-Unsafe"

Debugging Threading Issues

Helgrind (Valgrind tool for threading):

```
valgrind --tool=helgrind ./program
```

Detects: data races, lock ordering issues, misuse of POSIX API

ThreadSanitizer (TSan):

```
gcc -fsanitize=thread -g program.c -o program -pthread  
./program
```

Detects: data races, deadlocks at runtime

GDB thread commands:

```
info threads  
thread <n>
```

Section 3: Socket Programming

What are Sockets?

Socket: Communication endpoint for networking

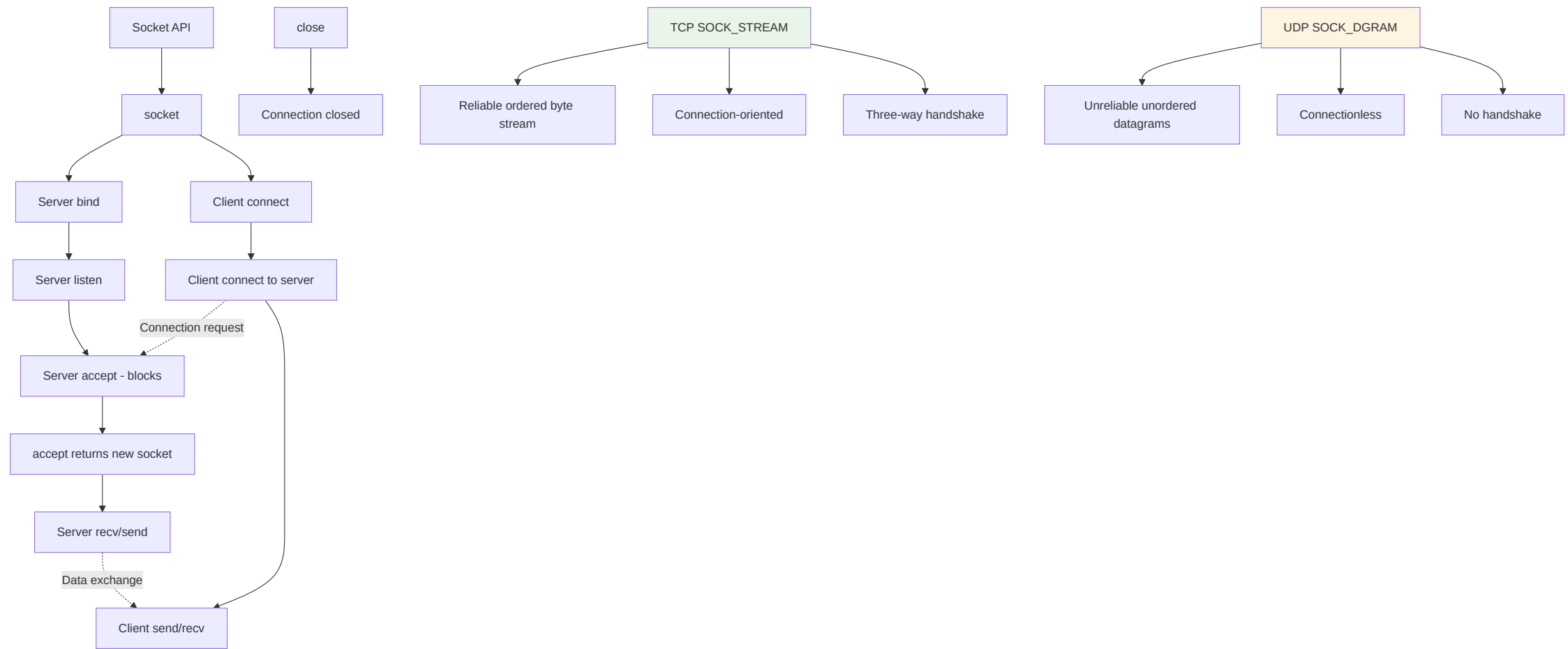
Socket Families:

- `AF_INET` : IPv4
- `AF_INET6` : IPv6
- `AF_UNIX` / `AF_LOCAL` : Unix domain sockets (local IPC)

Socket Types:

- `SOCK_STREAM` : TCP (reliable, connection-oriented)
- `SOCK_DGRAM` : UDP (unreliable, connectionless)
- `SOCK_RAW` : Raw IP packets (requires root)

Socket API Overview



Server: socket → bind → listen → accept → rcv/send → close

TCP vs UDP

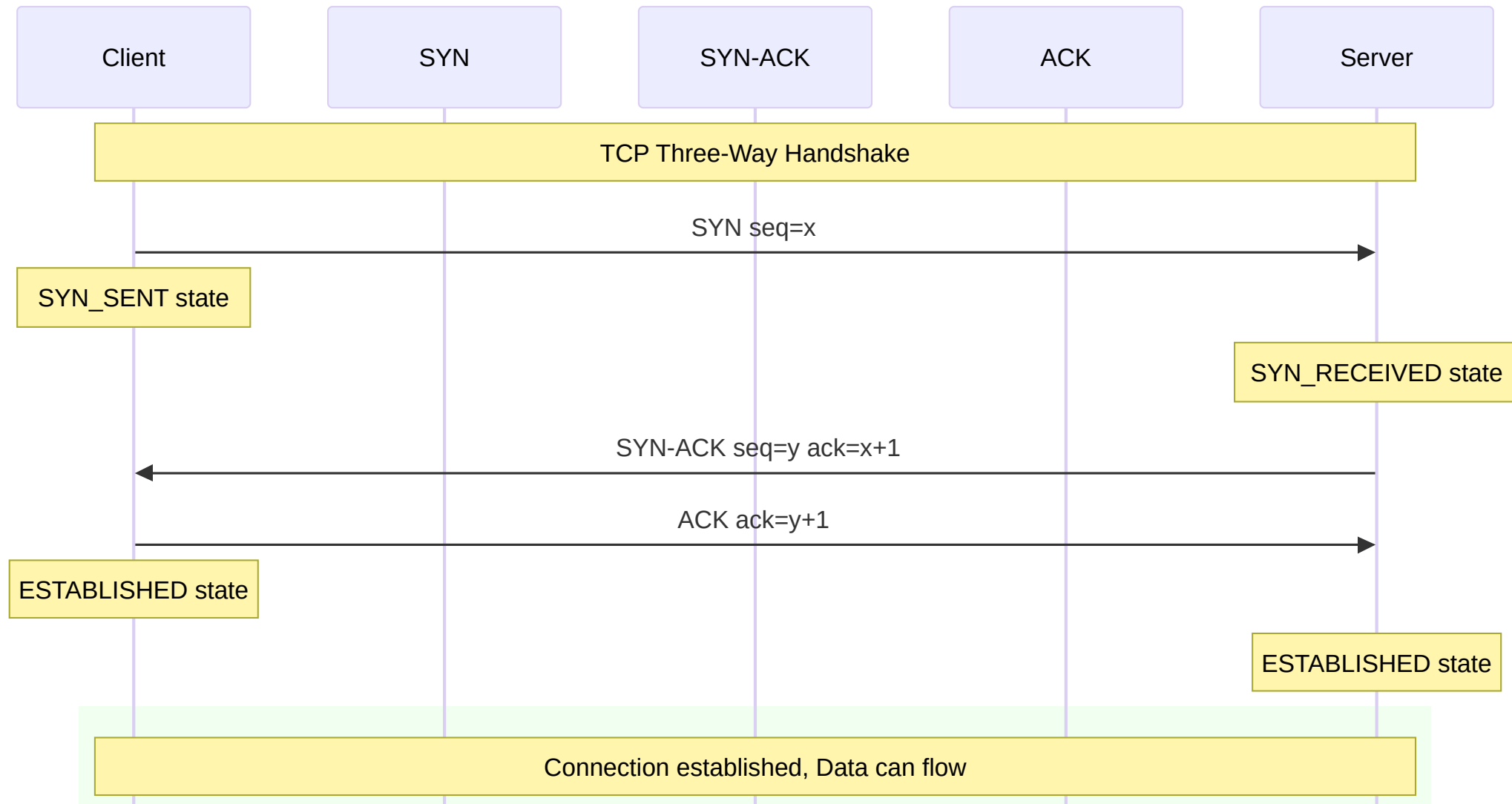
TCP (SOCK_STREAM):

- Connection-oriented (3-way handshake)
- Reliable, ordered delivery
- Flow control, congestion control
- Use for: HTTP, SSH, file transfer

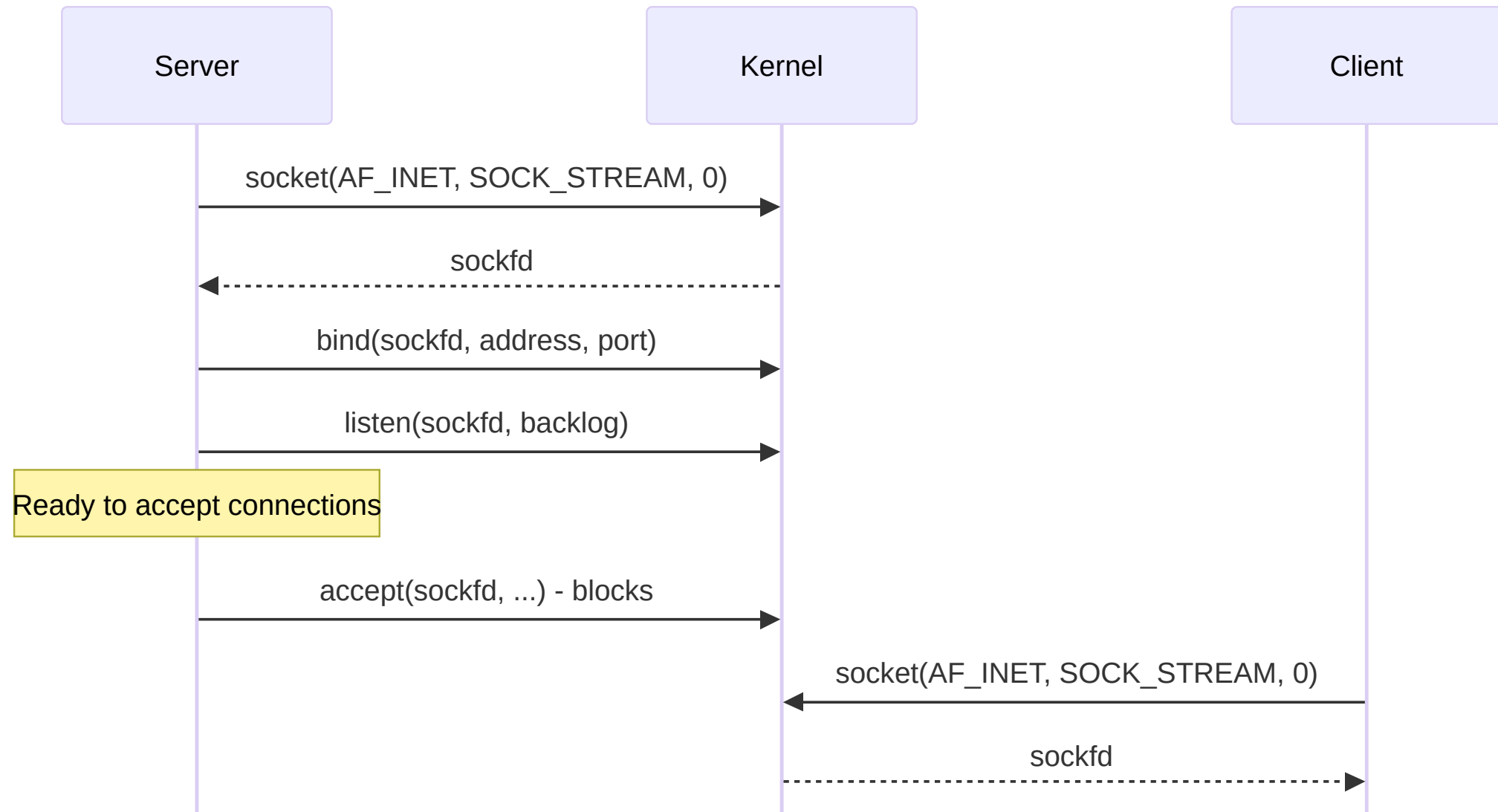
UDP (SOCK_DGRAM):

- Connectionless (no handshake)
- Unreliable, may lose/reorder packets
- Low latency, low overhead
- Use for: DNS, video streaming, gaming

TCP Three-Way Handshake



TCP Server Flow



Simple TCP Server

```
#include <sys/socket.h>
#include <netinet/in.h>
#include <string.h>
#include <unistd.h>

int main() {
    int server_fd, client_fd;
    struct sockaddr_in addr;

    // Create socket
    server_fd = socket(AF_INET, SOCK_STREAM, 0);

    // Bind to port 8080
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = INADDR_ANY;
    addr.sin_port = htons(8080);
    bind(server_fd, (struct sockaddr *)&addr, sizeof(addr));

    // Listen for connections
    listen(server_fd, 5);

    // Accept client
    client_fd = accept(server_fd, NULL, NULL);

    // Send response
    send(client_fd, "Hello!\n", 7, 0);

    close(client_fd);
    close(server_fd);
    return 0;
}
```

Simple TCP Client

```
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <string.h>

int main() {
    int sock_fd;
    struct sockaddr_in addr;
    char buf[100];

    // Create socket
    sock_fd = socket(AF_INET, SOCK_STREAM, 0);

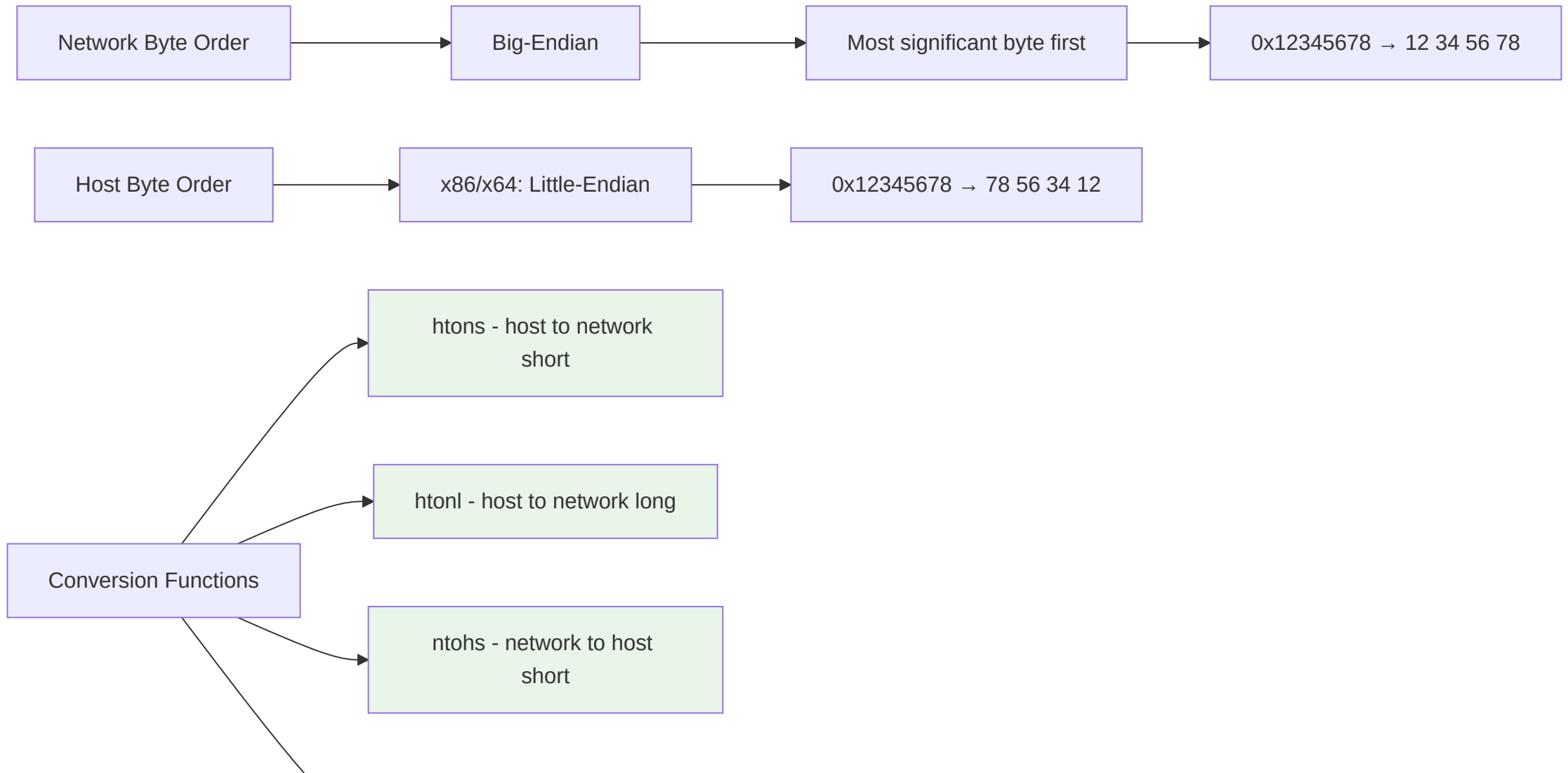
    // Connect to server
    addr.sin_family = AF_INET;
    addr.sin_port = htons(8080);
    inet_pton(AF_INET, "127.0.0.1", &addr.sin_addr);

    connect(sock_fd, (struct sockaddr *)&addr, sizeof(addr));

    // Receive response
    recv(sock_fd, buf, sizeof(buf), 0);
    printf("Received: %s", buf);

    close(sock_fd);
    return 0;
}
```


Network Byte Order



Byte Order Conversion

```
#include <arpa/inet.h>

uint16_t port = 8080;
uint32_t ip = 0xC0A80101; // 192.168.1.1

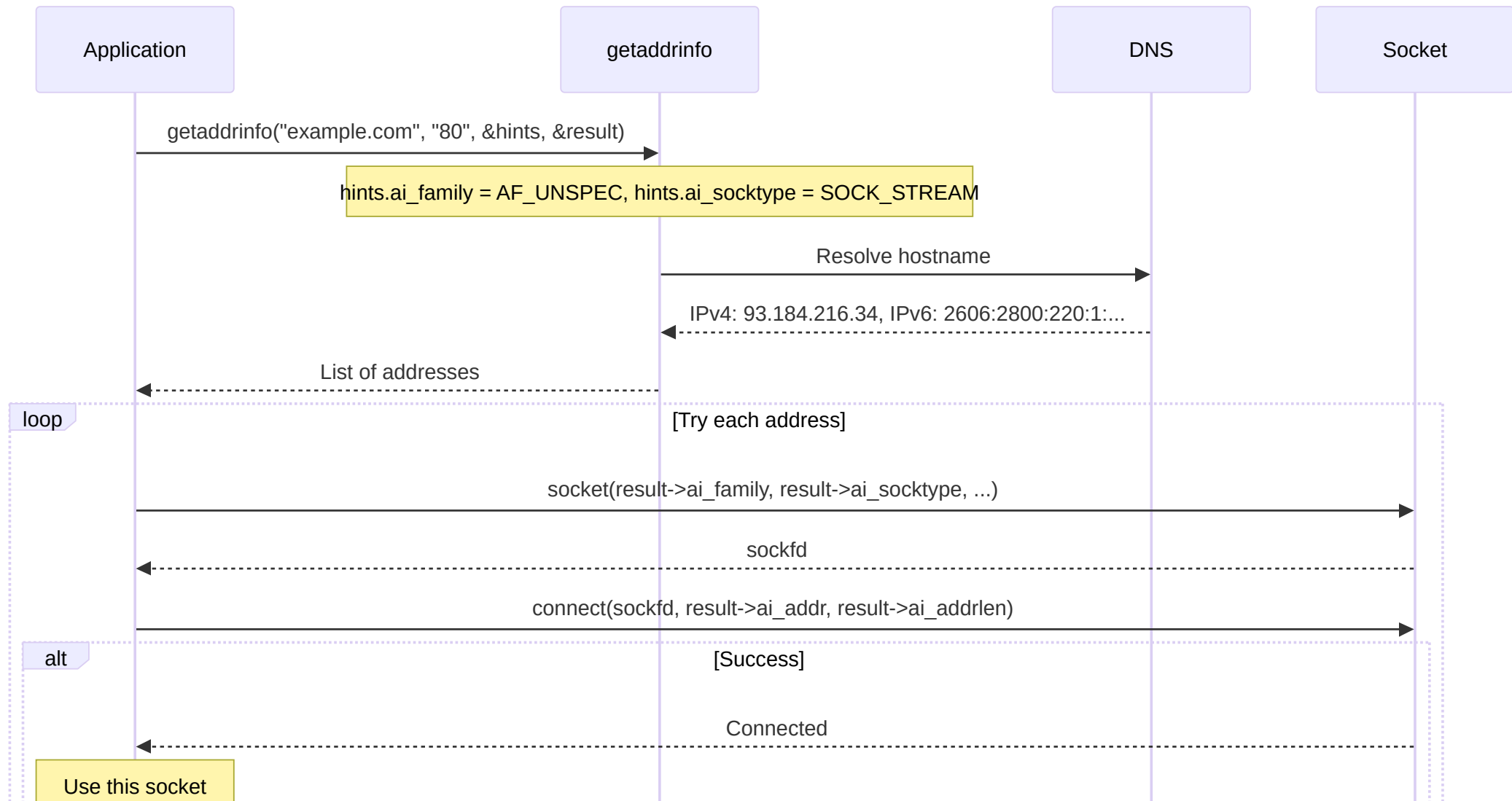
// Host to network
uint16_t net_port = htons(port); // host to network short
uint32_t net_ip = htonl(ip);      // host to network long

// Network to host
uint16_t host_port = ntohs(net_port); // network to host short
uint32_t host_ip = ntohl(net_ip);     // network to host long

// String to binary
struct in_addr addr;
inet_pton(AF_INET, "192.168.1.1", &addr); // Presentation to network

// Binary to string
char str[INET_ADDRSTRLEN];
inet_ntop(AF_INET, &addr, str, sizeof(str)); // Network to presentation
```

getaddrinfo() - Modern DNS Resolution



Using getaddrinfo()

```
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>

struct addrinfo hints, *result, *rp;
int sock_fd;

// Set criteria
memset(&hints, 0, sizeof(hints));
hints.ai_family = AF_UNSPEC;    // IPv4 or IPv6
hints.ai_socktype = SOCK_STREAM; // TCP

// Resolve hostname
getaddrinfo("example.com", "80", &hints, &result);

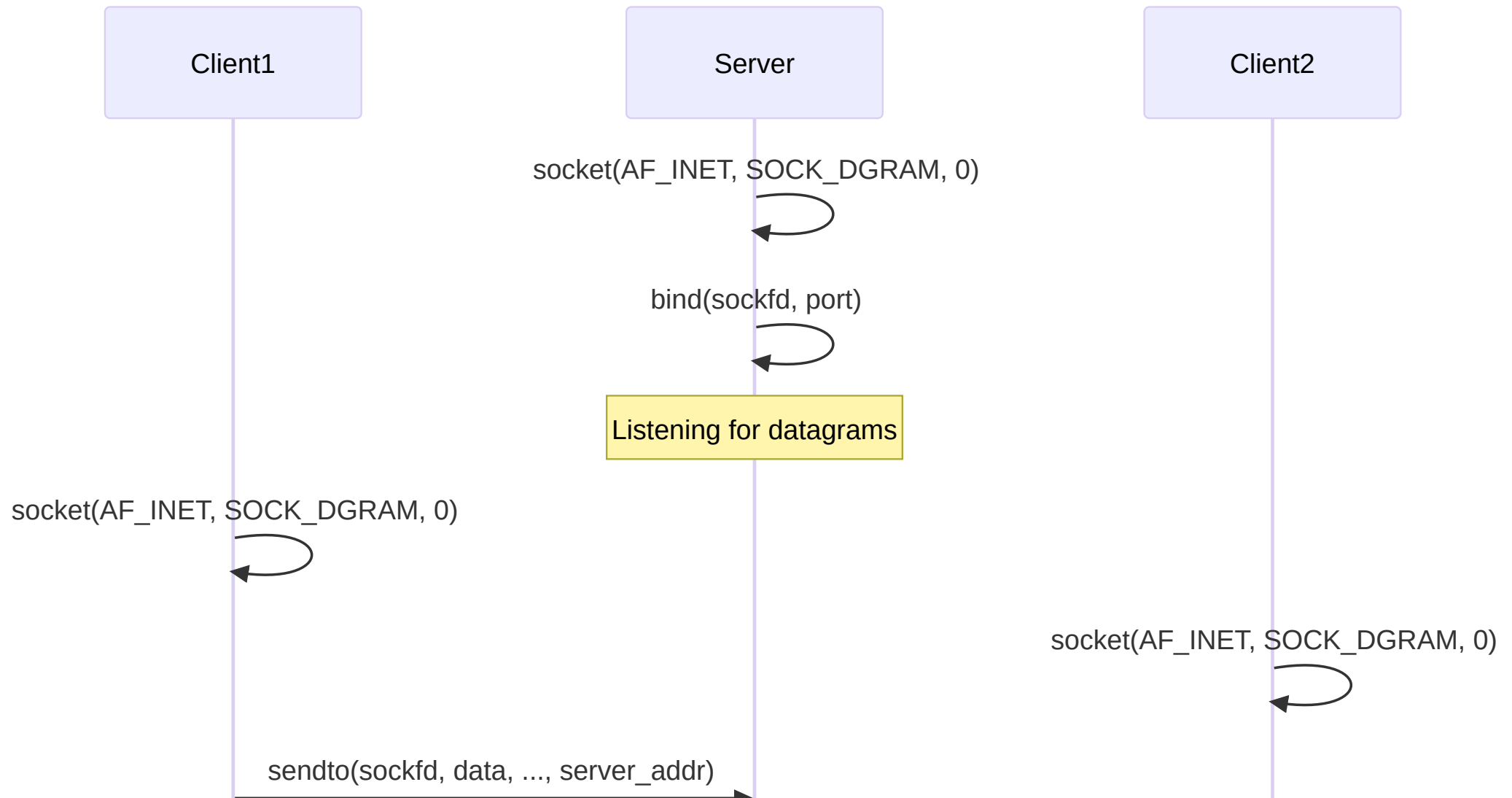
// Try each address
for (rp = result; rp != NULL; rp = rp->ai_next) {
    sock_fd = socket(rp->ai_family, rp->ai_socktype, rp->ai_protocol);
    if (sock_fd == -1) continue;

    if (connect(sock_fd, rp->ai_addr, rp->ai_addrlen) == 0)
        break;    // Success!

    close(sock_fd);
}

freeaddrinfo(result);
```

UDP Communication Flow



UDP Server

```
int sock_fd;
struct sockaddr_in server_addr, client_addr;
socklen_t client_len = sizeof(client_addr);
char buf[512];

// Create UDP socket
sock_fd = socket(AF_INET, SOCK_DGRAM, 0);

// Bind to port
server_addr.sin_family = AF_INET;
server_addr.sin_addr.s_addr = INADDR_ANY;
server_addr.sin_port = htons(9000);
bind(sock_fd, (struct sockaddr *)&server_addr, sizeof(server_addr));

// Receive datagram
ssize_t n = recvfrom(sock_fd, buf, sizeof(buf), 0,
                    (struct sockaddr *)&client_addr, &client_len);

// Send reply
sendto(sock_fd, "ACK", 3, 0,
       (struct sockaddr *)&client_addr, client_len);

close(sock_fd);
```

UDP Client

```
int sock_fd;
struct sockaddr_in server_addr;

// Create UDP socket
sock_fd = socket(AF_INET, SOCK_DGRAM, 0);

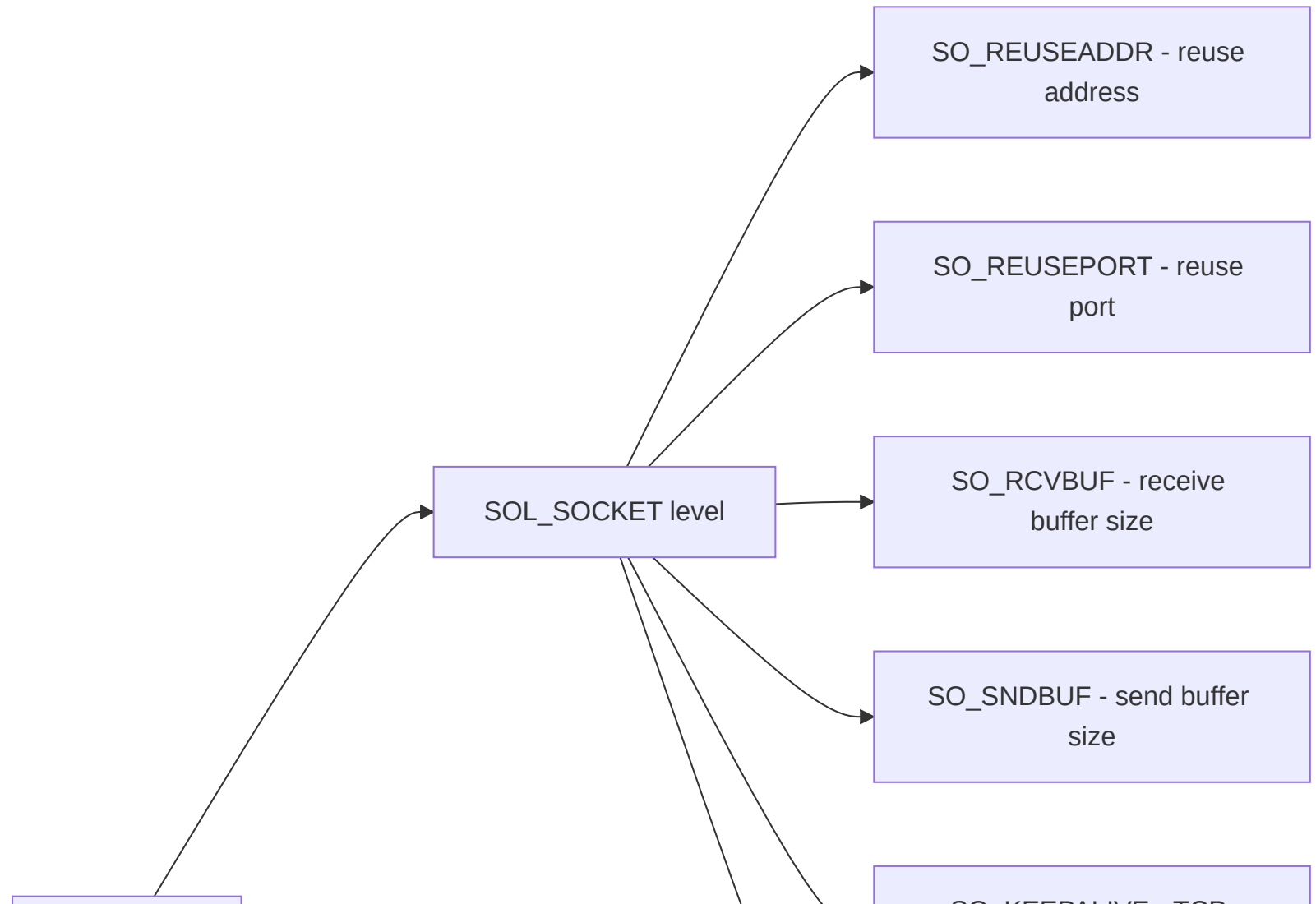
// Set server address
server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(9000);
inet_pton(AF_INET, "127.0.0.1", &server_addr.sin_addr);

// Send datagram (no connect needed!)
sendto(sock_fd, "Hello", 5, 0,
        (struct sockaddr *)&server_addr, sizeof(server_addr));

// Receive reply
char buf[512];
recvfrom(sock_fd, buf, sizeof(buf), 0, NULL, NULL);

close(sock_fd);
```

Socket Options



Setting Socket Options

```
int sock_fd = socket(AF_INET, SOCK_STREAM, 0);

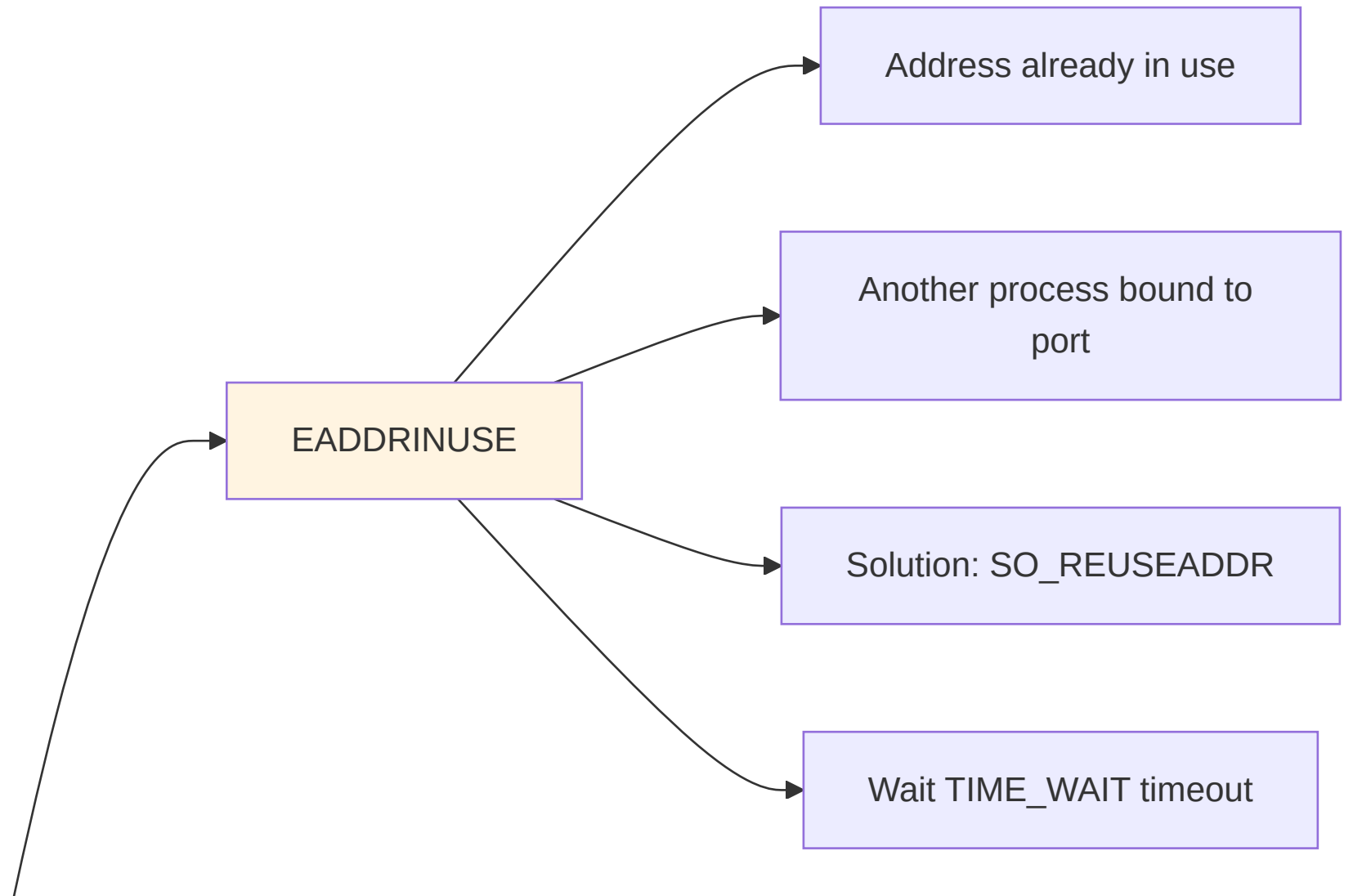
// SO_REUSEADDR: Reuse address immediately
int opt = 1;
setsockopt(sock_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

// SO_RCVBUF: Set receive buffer size to 64 KB
int bufsize = 64 * 1024;
setsockopt(sock_fd, SOL_SOCKET, SO_RCVBUF, &bufsize, sizeof(bufsize));

// TCP_NODELAY: Disable Nagle algorithm
int nodelay = 1;
setsockopt(sock_fd, IPPROTO_TCP, TCP_NODELAY, &nodelay, sizeof(nodelay));

// SO_KEEPALIVE: Enable TCP keepalive
int keepalive = 1;
setsockopt(sock_fd, SOL_SOCKET, SO_KEEPALIVE, &keepalive, sizeof(keepalive));
```

Common Socket Errors



Handling Socket Errors

```
// Ignore SIGPIPE (write to closed socket)
signal(SIGPIPE, SIG_IGN);

// Check send() return value
ssize_t n = send(sock_fd, buf, len, 0);
if (n < 0) {
    if (errno == EPIPE) {
        printf("Connection closed by peer\n");
    } else if (errno == EINTR) {
        // Interrupted by signal, retry
    } else {
        perror("send");
    }
}

// Retry on EINTR
while ((n = accept(server_fd, NULL, NULL)) < 0) {
    if (errno != EINTR) {
        perror("accept");
        break;
    }
}
```

Section 4: I/O Multiplexing

The C10K Problem

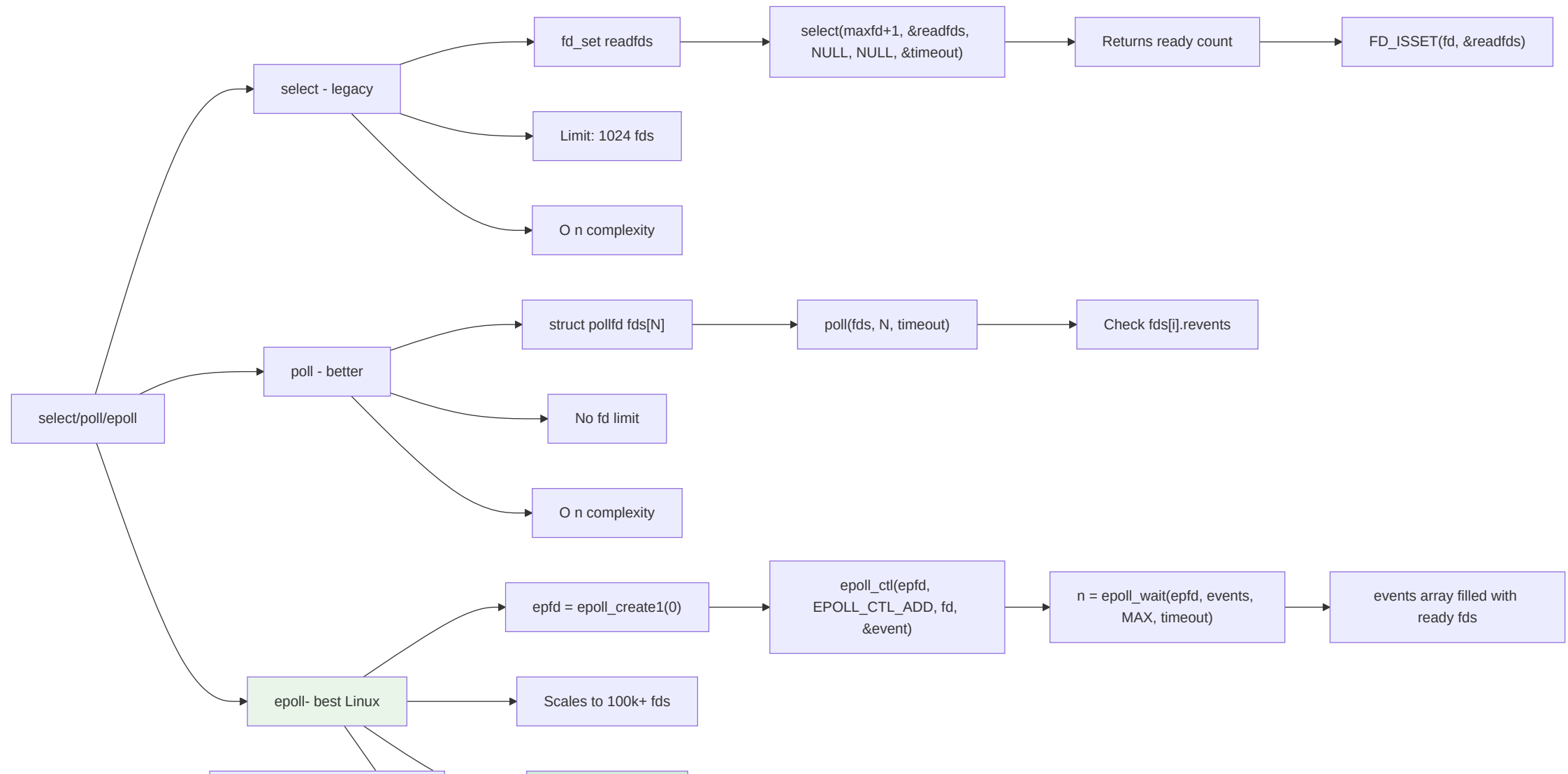
Problem: How to handle 10,000+ simultaneous connections?

Naive approaches fail:

- Thread-per-connection: 10,000 threads → excessive memory, context switches
- Fork-per-connection: Even worse than threads

Solution: I/O multiplexing with **select**, **poll**, or **epoll**

I/O Multiplexing Comparison



epoll Workflow

```
#include <sys/epoll.h>

// Create epoll instance
int epoll_fd = epoll_create1(0);

// Add server socket to epoll
struct epoll_event ev;
ev.events = EPOLLIN; // Monitor for read events
ev.data.fd = server_fd;
epoll_ctl(epoll_fd, EPOLL_CTL_ADD, server_fd, &ev);

// Event loop
struct epoll_event events[MAX_EVENTS];
while (1) {
    int n = epoll_wait(epoll_fd, events, MAX_EVENTS, -1);

    for (int i = 0; i < n; i++) {
        if (events[i].data.fd == server_fd) {
            // Accept new connection
            int client_fd = accept(server_fd, NULL, NULL);
            ev.events = EPOLLIN;
            ev.data.fd = client_fd;
            epoll_ctl(epoll_fd, EPOLL_CTL_ADD, client_fd, &ev);
        } else {
            // Handle client data
            handle_client(events[i].data.fd);
        }
    }
}
```

epoll Edge-Triggered vs Level-Triggered

Level-Triggered (default):

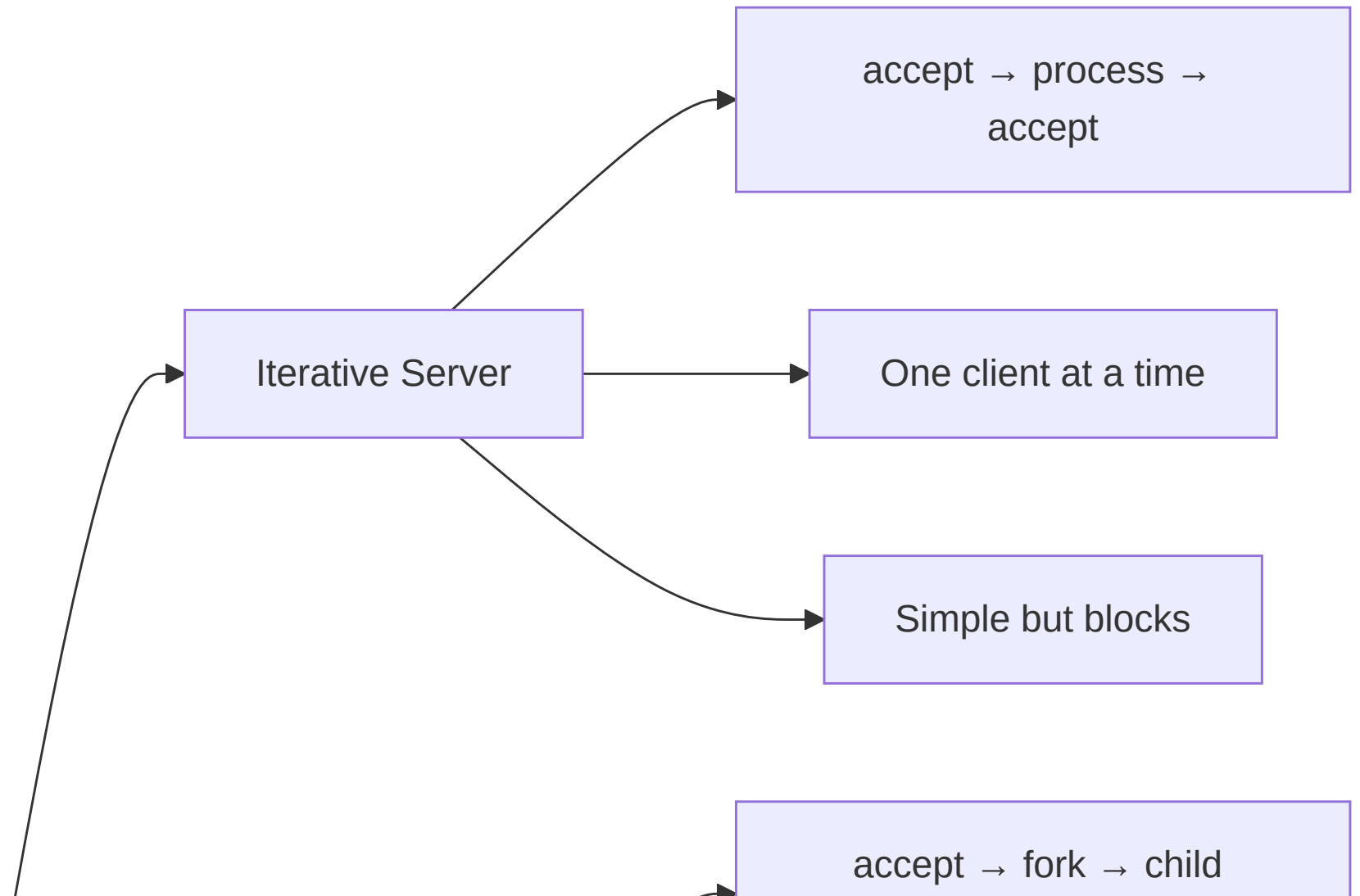
- Notification as long as condition is true
- `epoll_wait()` returns repeatedly if data available
- Easier to use

Edge-Triggered (EPOLLET):

- Notification only on state change
- Must read until `EAGAIN`
- More efficient, but harder to get right

```
// Edge-triggered + non-blocking
ev.events = EPOLLIN | EPOLLET;
fcntl(fd, F_SETFL, O_NONBLOCK);
```

Concurrent Server Architectures



Thread Pool Server

```
typedef struct {
    int client_fd;
} work_item_t;

pthread_mutex_t queue_mutex;
pthread_cond_t queue_cond;
work_item_t queue[MAX_QUEUE];
int queue_count = 0;

void *worker_thread(void *arg) {
    while (1) {
        pthread_mutex_lock(&queue_mutex);
        while (queue_count == 0)
            pthread_cond_wait(&queue_cond, &queue_mutex);

        work_item_t item = queue[--queue_count];
        pthread_mutex_unlock(&queue_mutex);

        handle_client(item.client_fd);
        close(item.client_fd);
    }
}

// Main thread accepts and enqueues
int client_fd = accept(server_fd, NULL, NULL);
pthread_mutex_lock(&queue_mutex);
queue[queue_count++] = (work_item_t){client_fd};
pthread_cond_signal(&queue_cond);
pthread_mutex_unlock(&queue_mutex);
```

Event-Driven epoll Server

```
int main() {
    int server_fd = create_and_bind("8080");
    listen(server_fd, SOMAXCONN);

    int epoll_fd = epoll_create1(0);
    add_to_epoll(epoll_fd, server_fd);

    struct epoll_event events[MAX_EVENTS];

    while (1) {
        int n = epoll_wait(epoll_fd, events, MAX_EVENTS, -1);

        for (int i = 0; i < n; i++) {
            if (events[i].data.fd == server_fd) {
                // Accept all pending connections
                while (1) {
                    int client_fd = accept(server_fd, NULL, NULL);
                    if (client_fd < 0) break;
                    set_nonblocking(client_fd);
                    add_to_epoll(epoll_fd, client_fd);
                }
            } else {
                handle_client_event(events[i].data.fd);
            }
        }
    }
}
```

Non-Blocking I/O

```
#include <fcntl.h>

// Set socket to non-blocking mode
int flags = fcntl(sock_fd, F_GETFL, 0);
fcntl(sock_fd, F_SETFL, flags | O_NONBLOCK);

// Now read() returns immediately with EAGAIN if no data
ssize_t n = read(sock_fd, buf, sizeof(buf));
if (n < 0) {
    if (errno == EAGAIN || errno == EWOULDBLOCK) {
        // No data available, try later
    } else {
        perror("read");
    }
}

// Use with epoll edge-triggered mode
while (1) {
    ssize_t n = read(sock_fd, buf, sizeof(buf));
    if (n < 0) {
        if (errno == EAGAIN) break; // All data read
        perror("read");
        break;
    }
    process(buf, n);
}
```

Section 5: Best Practices & Summary

Threading Best Practices

- ✓ **Always** use mutexes for shared data
- ✓ **Always** call `pthread_join()` or detach threads
- ✓ **Always** use `while` loop with condition variables (spurious wakeups)
- ✓ **Always** check function return values
- ✓ Use lock hierarchy to prevent deadlocks
- ✓ Use reader-writer locks for read-heavy workloads
- ✓ Use TLS for per-thread state
- ✓ Use thread sanitizer (`-fsanitize=thread`) during development

Socket Programming Best Practices

- ✓ **Always** use `SO_REUSEADDR` for servers
- ✓ **Always** convert to network byte order with `htons()` / `htonl()`
- ✓ **Always** handle `EINTR` (interrupted system calls)
- ✓ **Always** ignore `SIGPIPE` or handle `EPIPE`
- ✓ Use `getaddrinfo()` instead of deprecated functions
- ✓ Use `inet_pton()` / `inet_ntop()` for address conversion
- ✓ Use `epoll` for scalable servers (1000+ connections)
- ✓ Use non-blocking I/O with `epoll` edge-triggered mode
- ✓ Set timeouts on `recv()` / `send()` with `SO_RCVTIMEO` / `SO_SNDTIMEO`

Common Mistakes

- ✗ **Race conditions:** Forgetting mutex protection
- ✗ **Deadlocks:** Inconsistent lock ordering
- ✗ **Memory leaks:** Not joining threads, not calling `freeaddrinfo()`
- ✗ **Partial I/O:** Not handling partial `send()` / `recv()`
- ✗ **Byte order:** Forgetting `htons()` / `htonl()`
- ✗ **EINTR:** Not retrying after interrupted system calls
- ✗ **Using non-thread-safe functions:** Use `_r` variants
- ✗ **Sharing stack variables:** Passing local var address to threads

Key Takeaways

Threading

- Threads share memory but need synchronization (mutexes, condition variables)
- Always prevent race conditions and deadlocks
- Use thread-safe functions or `_r` variants
- Detach threads or join them to avoid leaks

Socket Programming

- TCP (SOCK_STREAM): reliable, connection-oriented
- UDP (SOCK_DGRAM): fast, connectionless, unreliable
- Always convert to network byte order
- Use `getaddrinfo()` for DNS and IPv6 support

Performance Tips

Threading:

- Use atomic operations for simple counters (faster than mutexes)
- Reader-writer locks for read-heavy workloads
- Thread pool to avoid thread creation overhead
- Avoid excessive locking (coarse-grained vs fine-grained)

Sockets:

- Use `TCP_NODELAY` for low-latency protocols
- Use `SO_RCVBUF` / `SO_SNDBUF` to tune buffer sizes
- Use `sendfile()` for zero-copy file transmission
- Use epoll edge-triggered mode with non-blocking I/O
- Set `TCP_CORK` to aggregate small writes

Debugging Tools

Threading:

```
gcc -pthread -g -fsanitize=thread program.c
valgrind --tool=helgrind ./program
gdb: info threads, thread apply all bt
```

Sockets:

```
netstat -tulpn      # List listening ports
ss -s              # Socket statistics
lsof -i :8080       # What's using port 8080
tcpdump -i lo       # Capture network traffic
strace -e trace=network ./program
```

Lab Exercises - Day 5

Exercise 1: Multi-threaded Web Server

Implement a concurrent web server:

- Accept HTTP requests on port 8080
- Use thread pool (4 workers) to handle requests
- Serve static files from `./www` directory
- Return proper HTTP responses (200 OK, 404 Not Found)

Exercise 2: Chat Application

Build a TCP chat server and client:

- Server uses epoll to handle multiple clients

Lab Exercises (Continued)

Exercise 3: Producer-Consumer Pipeline

Create a multi-stage data processing pipeline:

- Stage 1: Read lines from file (producer thread)
- Stage 2: Process/transform data (3 worker threads)
- Stage 3: Write results (consumer thread)
- Use condition variables and work queues
- Measure throughput

Exercise 4: UDP Port Scanner

Implement a threaded UDP port scanner:

Day 5 Complete!

What We Covered

- ✓ **POSIX Threads:** creation, joining, detaching, attributes
- ✓ **Thread Synchronization:** mutexes, condition variables, deadlocks
- ✓ **Thread Safety:** TLS, race conditions, atomic operations
- ✓ **Socket Programming:** TCP/UDP clients and servers
- ✓ **Network Byte Order:** htons/htonl, getaddrinfo
- ✓ **I/O Multiplexing:** select/poll/epoll, event-driven servers
- ✓ **Concurrent Architectures:** thread pools, event loops

Course Complete! 🎉

5-Day Journey Recap

Day 1: Build Tools & GCC

Day 2: Debugging & System Calls

Day 3: File Systems & I/O

Day 4: Processes & IPC

Day 5: Threading & Networking

You now have the skills to build:

- High-performance multi-threaded applications
- Scalable network servers (C10K+)
- Concurrent data processing pipelines

Next Steps

Continue Learning

- **Books:** "The Linux Programming Interface" by Michael Kerrisk
- **Practice:** Contribute to open source projects
- **Explore:** Linux kernel internals, eBPF
- **Build:** Your own projects using these skills

Advanced Topics

- Asynchronous I/O (io_uring)
- Memory-mapped files for IPC
- Lock-free data structures
- Network protocol design

Resources

Documentation

- `man 7 pthreads` - POSIX threads overview
- `man 7 socket` - Socket interface overview
- `man 7 tcp` - TCP protocol
- `man 7 udp` - UDP protocol
- `man 7 epoll` - epoll interface

Online

- <https://man7.org/linux/man-pages/> - Linux man pages
- <https://github.com/torvalds/linux> - Linux kernel source
- <https://gist.github.com/ihoner/2841832> - Latency numbers

Thank You!

Questions?

Contact: [Instructor Email]

Course Materials: <https://github.com/it-scholar/linux/>

Feedback

Please provide feedback on:

- Course content and pacing
- Exercises and difficulty
- Topics you'd like to explore further

Keep learning and building great software! 

Summary

Key Takeaways

- POSIX threads and synchronization primitives
- Socket programming patterns (client/server)
- epoll for scalable I/O multiplexing
- Concurrent server architectures

Course Complete!

You've Learned

- Complete Linux application development toolchain
- System programming interfaces
- Process and thread management
- Network programming

Continue practicing with the Weather Station Project!

See exercises: `05-threads-sockets-servers.md`